Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

Fourier and applications

1. REMOVE NOISE TO LENA

1.1. Introduction

In this exercise, we are asked to remove the noise from the image "lena_noisy.png" (Figure 1) and make it look like the original photo "lena.png" (Figure 2) after applying this transformation.

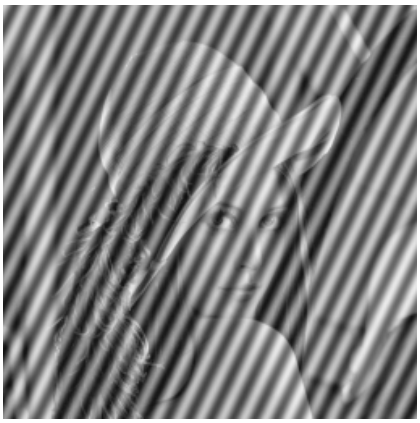

Figure 1: Image of Lena with the noise

Figure 2: Image of Lena with the noise

1.2. First intuition

Our first thought was to match the noise seen in the Lena image with an example explained in the theory (Figure 3). The noise from the theory class had a similar shape, appearing as a black and white wave. The theory included a plot of this noise in the frequency domain (Figure 4), which could simply be described as two dots specifying the frequency values of the noise. So, our first hypothesis was that the aim of the exercise was to find the dots in the frequency domain of Figure 1 and then apply a filter to remove them, hoping this would achieve the goal of the exercise.

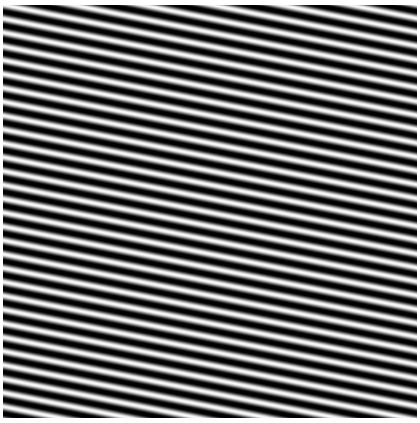

Figure 3: Example of noise explained in the theories

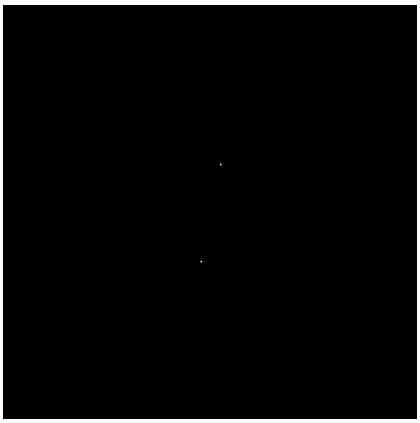

Figure 4: Frequency domain of Figure 3

1.3. First look at our image

To confirm that we were right we applied a FFT (Fast Fourier Transform) to Figure 1 (Figure 5) specting that the result would have the same frequency domain as Figure 1 (Figure 6) but with two dots representing the noise.

Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

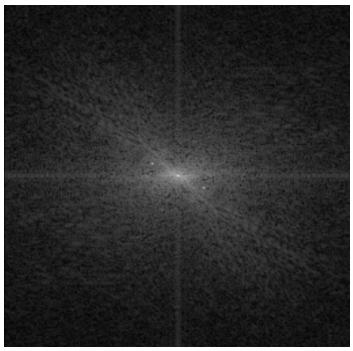

Figure 5: FFT of the noisy Lena image

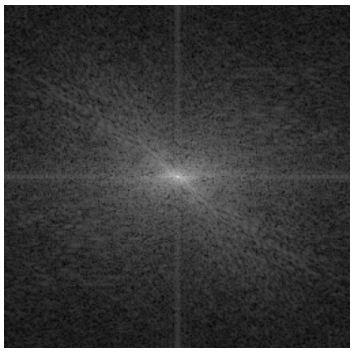

Figure 6: FFT of the original Lena image

As we can see, the result is exactly as we expected. Now thw next step would be remove this two dots from the frequency domain of Figure 1 and then apply an inverse FFT to get the resulting image.

1.4. First intuition of the filter

Taking into account where were places the two dots of Figure 5 we thought that the proper filter explained in the theory should be the BPF (Band Pass Filter). The dots were representing a low frequency respect to some other that could be seen in the original image, but they were not even close the lowest ones. So applying a combination of filters that remove lower or equal frequencies as the frequencies of the dots was a bad idea, because we would be removing a lot of frequencies that are not noise. And the image could be too blurry. On the other hand, applying a BPF that only remove frequencies around the ones of the dots would be a good idea, because we would be removing only the noise and not the rest of the frequencies.

1.5. Creation of the filter

To create the filter was really helpful to learn from the filter examples in the Colab NoteBook. After analyzing some of them we decided that our filter should the sum of a LPF and HPF. In this way, the plot result of the filter should be at most white, but it should have a black circle in the middle representing the frequencies that we want to remove. The rest of the frequencies should be white, because we want to keep them. With the example filters we did a first plot representing the first idea we had for the filter (Figure 7).

We had a lot of trubble to find the right values for the radius of the circles of the LPF and HPF, but after some trial and error we found that a radius of 20 pixels for the LPF and 21.3 pixels for the HPF were strictly the right values to remove the noise without removing too much frequencies. The resulting filter can be seen in Figure 8.

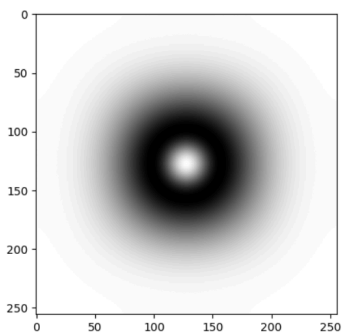

Figure 7: First idea of the filter

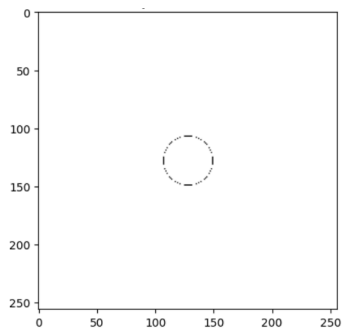

Figure 8: Resulting filter

Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

1.6. Application of the filter

After creating the filter, we applied it to the FFT of Figure 1 and then we applied an inverse FFT to get the resulting image. The resulting image can be seen in Figure 9. As we can see, the resulting image is really similar to the original Lena image, but it is not exactly the same. We can see that there is still some noise in the resulting image, but it is really reduced compared to the original noisy image. We think that the reason why there is still some noise in the resulting image is because the combination of the two filters is not perfect, due to its circular shape. The ideal filter should be two circles with the same radius as the dots and centered in the same position as the dots. But with the resources given in the theoretical and practical classes, we consider we were not able to create a filter with this shape, so we had to settle for a combination of two filters that was not perfect, but it was good enough to remove most of the noise.

Figure 9: Resulting image after applying the filter

1.7. Extra: Asking AI for the perfect filter

To understand well how our ideal filter would be implemented, we asked to an AI to create a filter that would remove the noise from the image. AI talked about a Notch Filter which was not in the theory classes but had the exactly shape we thought it would have. The resulting filter can be seen in Figure 10. It is interesting to see the plot of the filter applied in the frequency domain of the noisy image (Figure 11). And finally we can observe that the final result it is almost perfect and can be seen in Figure 12. The only downside is that the lines and they can still be slightly detected.

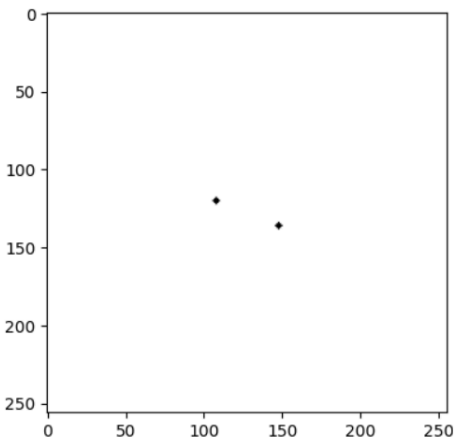

Figure 10: Ideal filter created by AI

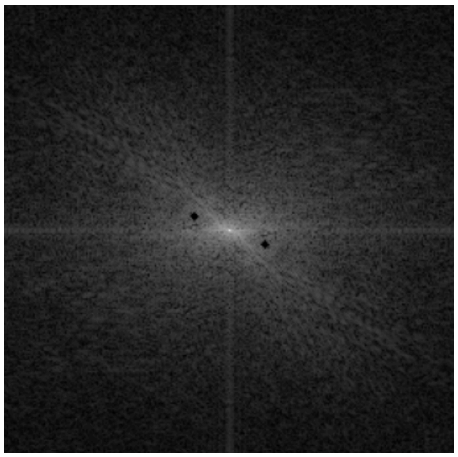

Figure 11: FFT of the noisy image with the ideal filter applied

Figure 12: Resulting image after applying the ideal filter

Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

2. SUBSAMPLING AND ALIASING

2.1. Introduction

In this exercise, we study what happens when an image with thin patterns is subsampled. We used the image of the striped pants (Figure 13), because the stripes are a good example of high frequency content. The exercise consists in reducing the image by different factors and comparing the result with and without pre-processing.

Figure 13: Original image used for the subsampling exercise

2.2. Pre-processing to avoid aliasing after subsampling

Before shrinking the image, we applied a low-pass filtering step. This is necessary because the image contains many small details, especially the stripes on the pants, and these details correspond to high frequencies. From the theory, we know that aliasing often appears in images with this kind of content, for example brick walls, striped clothes, or repetitive textures.

When the image is made smaller without any previous filtering, these high frequencies cannot be represented correctly in the reduced image. As a consequence, they appear as strange patterns or distorted textures. For this reason, the image has to be smoothed before subsampling. This reduces or removes the frequencies that would not fit correctly in the smaller image. In our case, this low-pass filtering step helps the stripes of the pants look more stable after subsampling.

2.3. Pre-processing for factor 2 and factor 4

For subsampling by a factor of 2, we used a smoothing filter before taking every other pixel. In our case, we applied a 3x3 average filter. This works because the reduction is not too strong. The image loses some detail, but a light blur is enough to remove many of the high frequencies that would cause aliasing.

The theory explains subsampling of order 2 as keeping samples such as f_{2k}. In the Fourier domain, this operation combines coefficients from different frequency regions. This is the reason why filtering is needed before reducing the image: if the high frequencies are not removed first, they can be mixed with lower frequencies and create aliasing artifacts.

Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

For subsampling by a factor of 4, the filtering has to be stronger. The image is reduced much more, so more high-frequency information must be removed before applying the subsampling. In this case, we used a 5x5 average filter. The result is more blurred than with a factor of 2, but this is expected. The purpose of the filter is to remove details that would later become aliasing artifacts. In general, the larger the subsampling factor, the stronger the low-pass filtering has to be.

2.4. Ringing

In our results, ringing does not appear clearly. Ringing is related to the Gibbs phenomenon, which appears as oscillations around sharp discontinuities. In images, it can be seen as bands or ripples near edges, especially when high frequencies are removed too abruptly.

In this experiment, we used an average filter in the spatial domain, not an ideal sharp cut-off filter in the Fourier domain. Because of that, the ringing effect is not very noticeable. If ringing appeared, it could be reduced by using smoother filters, for example a Gaussian filter, instead of using a very abrupt binary mask in the Fourier domain. The lab material also explains that binary masks can produce ringing because they are not smooth, while Gaussian masks reduce this effect better.

3. (ADD TITLE)

4. ANALYSIS OF IMAGE ROTATION

4.1. Introduction

Once read the statement of the execise we observe familiar topics that were explained in the theory lessons. The first part of the exercise asks us to implement a shear operator, described by the transformation (x, y)⟶ (x+ay,y). This is a type of affine transformation that shifts the x-coordinate of each pixel by an amount proportional to its y-coordinate. The second part of the exercise asks us to implement Yaroslavski's method for rotating an image, which expresses a rotation as a product of three shear operations.

4.2. Shear operator

To implement the shear operator we could simply shift each row by the fractional amount given by the parameter a. But the problem here is that we would get a blurry image and a lost in the high frequencies. The reason for that is that this method requires interpolating pixel values, which is not ideal for this kind of transformation. Instead, as the most of the cases in the transformation of images, it is better to apply the transformation in the Fourier domain. In this case, we can take into account the Fourier Shift Theorem, which states that a shift in the spatial domain corresponds to a multiplying it by a linear phase factor in the frequency domain. The process would be:

1. Apply FFT to convert the image to the frequency domain
2. Multiply each frequency component u of row y by the phase shift $e^{\mathrm{j}2\pi \mathrm{u} \cdot \frac{a \cdot y}{N}}$
3. Apply the 1D Inverse FFT (IFFT) to get the perfectly shifted row back in the spatial domain.

It will be easier to understand this second method if we plot the spatial domain for a simple image, for example for Figure 13. In the next section we will see that for applying the Yaroslavski's method we will need to apply the shear operator three times, where two of them will be an horizontal shear and the other one will be a vertical shear. So, for the horizontal shears we will apply the method explained above, and for the vertical shear we will apply the same method

Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

but shifting columns instead of rows. In Figure 14 we can see the result of applying the horizontal shear operator to the image of the pants with a value of a equal to 0.4. And in Figure 15 we can see the result of applying the vertical shear operator to the image of the pants with a value of a equal to 0.4.

Figure 14: Horizontal shear operator applied to the image of the pants with a value of a equal to 0.4

Figure 15: Vertical shear operator applied to the image of the pants with a value of a equal to 0.4

4.3. Yaroslavski's method for rotation

A standard 2D rotation matrix requires calculating new (x, y) coordinates for every pixel, which demands computationally heavy and lossy 2D interpolation. Yaroslavski's method decomposes a rotation into three shear operations, which can be implemented more efficiently. The steps for a rotation by an angle θ are:

1. Shear horizontally by $a = -\tan\left(\frac{\theta}{2}\right)$
2. Shear vertically by $b = \sin(\theta)$
3. Shear horizontally again by $c = -\tan\left(\frac{\theta}{2}\right)$

For a rotation of $\frac{\pi}{2}$ radians, the values of the shears would be: a = -tan(pi/4) = −1 b = sin(pi/2) = 1 c = -tan(pi/4) = −1.

Applying these three shears in sequence will rotate the image by 90 degrees. The advantage of this method is that each shear can be implemented using 1D interpolation, which is computationally more efficient than 2D interpolation required for a direct rotation. After applying Yaroslavski's method, we can compare the result with a naive rotation method that directly calculates the new coordinates for each pixel and uses 2D interpolation, or with a rotation done using a drawing program. The results should be similar, but Yaroslavski's method should be more efficient and may produce less blurring due to the use of 1D interpolation.

4.4. Results

For the last part of the exercise, we applied Yaroslavski's method to rotate the image of the pants by 90 degrees. The resulting image can be seen in Figure 16. We also applied a naive rotation method that directly calculates the new coordinates for each pixel and uses 2D interpolation, and the result can be seen in Figure 17.

Jan Amat Pallejà u251324
Guillem Esplugas Martínez u251682

Figure 16: Yaroslavski's method of rotating the image of the pants by 90 degrees

Figure 17: Naive rotation of the image of the pants by 90 degrees

At first we can not notice any difference between the two resulting images, that is because only one rotation is not sufficietly to see the difference. So we applied the same rotation fifty times in a row, for an angle of 360/50 = 7.2 degrees that would result in a full rotation. The resulting images can be seen in Figure 18 and Figure 19.

Figure 18: Yaroslavski's method of rotating the pants by 7.2 degrees fifty times in a row

Figure 19: Naive rotation of the pants by 7.2 degrees fifty times in a row

Now we can see that the resulting image of Yaroslavski's method is really similar to the original image, while the resulting image of the naive rotation method is really blurry and it is hard to recognize the original image. Hence, we can conclude that Yaroslavski's method is more efficient and produces less blurring than the naive rotation method, especially when applying multiple rotations in sequence.